



# 第四讲 计算机构造原理(下)

南开大学·软件学院  
姜桂飞

G.Jiang@nankai.edu.cn

# 复习回顾

1. 各种数据 7+3
2. 二进制数据 0 0111  
+ ) 0 0011

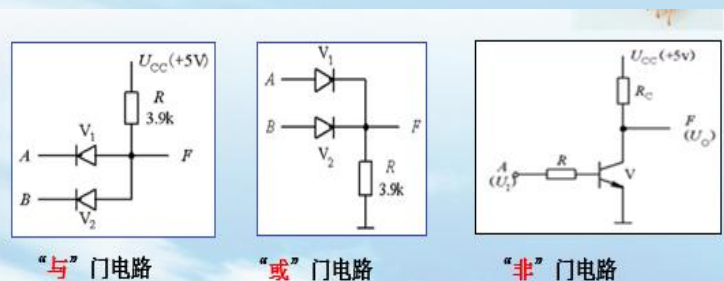
3.

布尔运算



- 基本逻辑运算
  - 非
  - 与
  - 或
- 混合逻辑运算
  - 同或 异或
  - 与非
  - 或非

4.



## 计算机为什么能计算？

◆ 0/1与数据表示

◆ 0/1计算与布尔运算

◆ 布尔运算与门电路



# 内容概要

- 计算机为什么能计算？
  - 0/1与数据表示
  - 0/1计算与布尔运算
  - 布尔运算与门电路
- 什么能被有效地计算？
  - 图灵机的思想与模型
- 现代计算机的体系构成
  - 冯·诺依曼机思想与结构



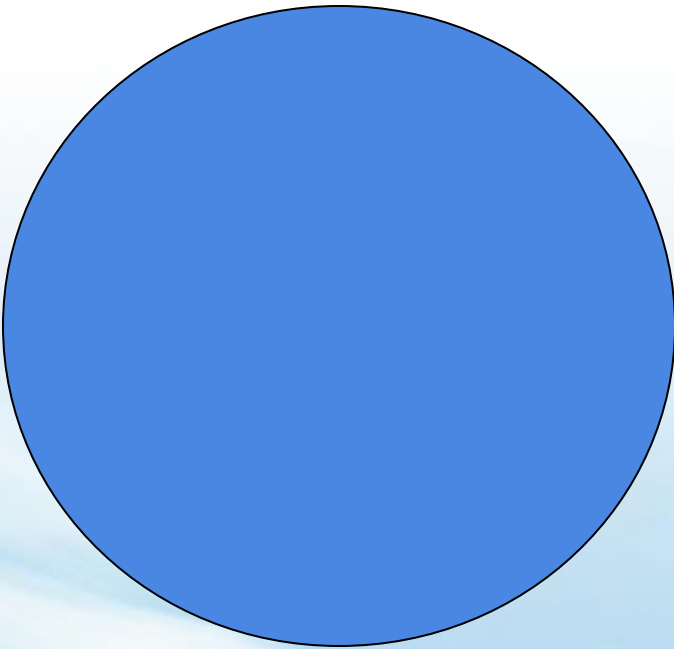
## 目标二

- 计算机为什么能计算？
  - 0/1与数据表示
  - 0/1计算与布尔运算
  - 布尔运算与门电路
- 什么能被有效地计算？
  - 图灵机的思想与模型
- 现代计算机的体系构成
  - 冯·诺依曼机思想与结构



# 什么能被自动有效地计算？

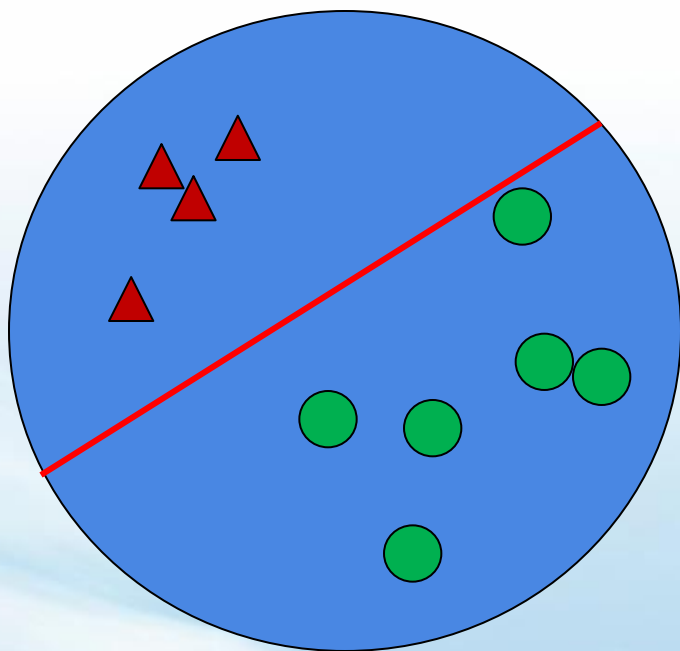
- 是不是所有的问题？





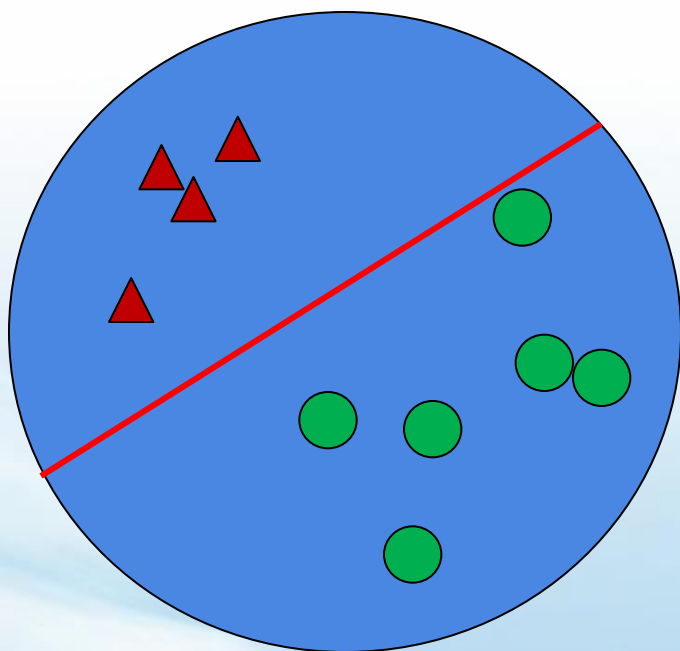
# 什么能被自动有效地计算？

- 是不是所有的问题？
- 不是的话，界限在哪儿？





# 什么能被自动地计算？



## ■ 可计算问题

- 设函数 $f$ 的定义域是 $D$ ，值域是 $R$ ，如果存在一个算法，对 $D$ 中任意给定 $x$ ，都能计算出 $f(x)$ 的值，则称函数 $f$ 是**可计算的**。

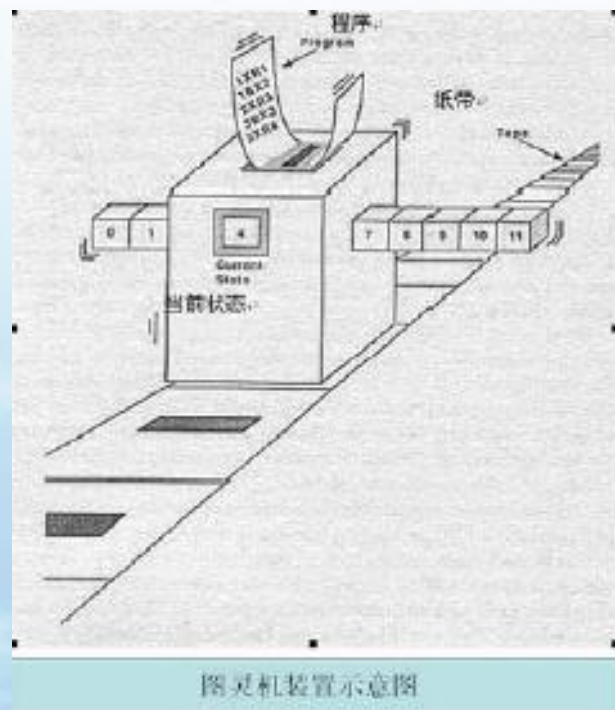
## ■ 解决思路

- 为计算建立一个数学模型，称为**计算模型**，然后证明凡是这个计算模型能完成的任务，就是可计算的任务。



# 图灵与图灵机

- ◆ 英国数学家、逻辑学家**阿兰·图灵**(Alan Turing, 1912~1954)
- ◆ 1936 年，图灵在其著名的论文《论可计算数及其在判定问题中的应用》中提出了一种理想的计算机器的数学模型——**图灵机 (Turing Machine)**

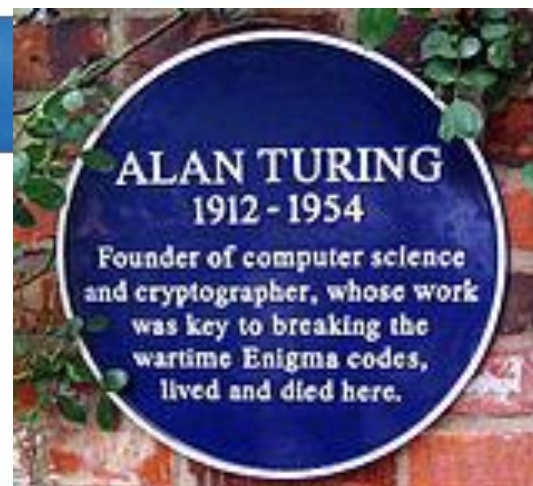


图灵机装置示意图





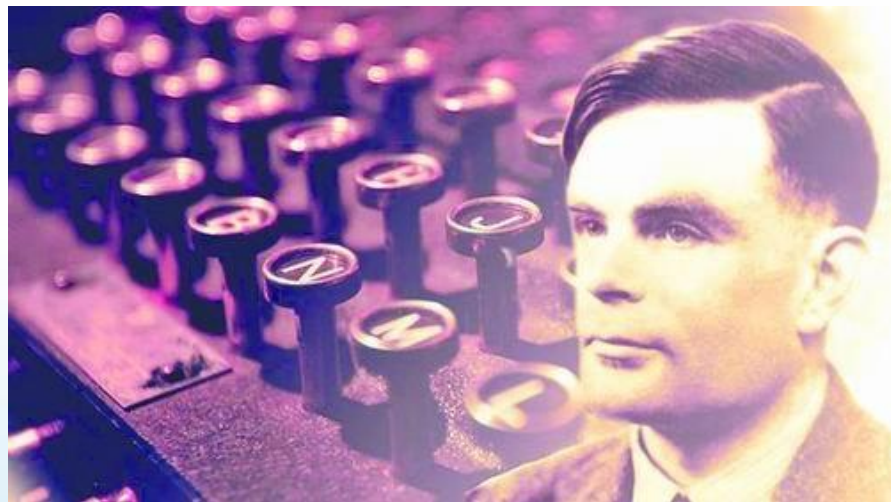
## 图灵其人



- ◆ 1912年6月出生于英国伦敦
- ◆ 中学期间，获得国王爱德华六世数学金盾奖章
- ◆ 1931年入剑桥国王学院
- ◆ 1935年当选为国王学院研究员
- ◆ 1936年5月，提出图灵机，发表在《伦敦数学会文集》
- ◆ 1938年在美国普林斯顿大学获博士学位
- ◆ 1938-1945年二战期间，密码破译工作
- ◆ 1945年获政府的最高奖——大英帝国荣誉勋章
- ◆ 1950年，提出“图灵测试”，发表论文《机器能思考吗》，人工智能领域的开山之作
- ◆ 1951年，被评为英国皇家学会会员（家族中第四位）
- ◆ 1952年，写了一个国际象棋程序
- ◆ 1954年，逝世

# 图灵奖

- 美国计算机协会 (ACM) 于1966年设立了计算机学界最高荣誉奖：  
**图灵奖**。



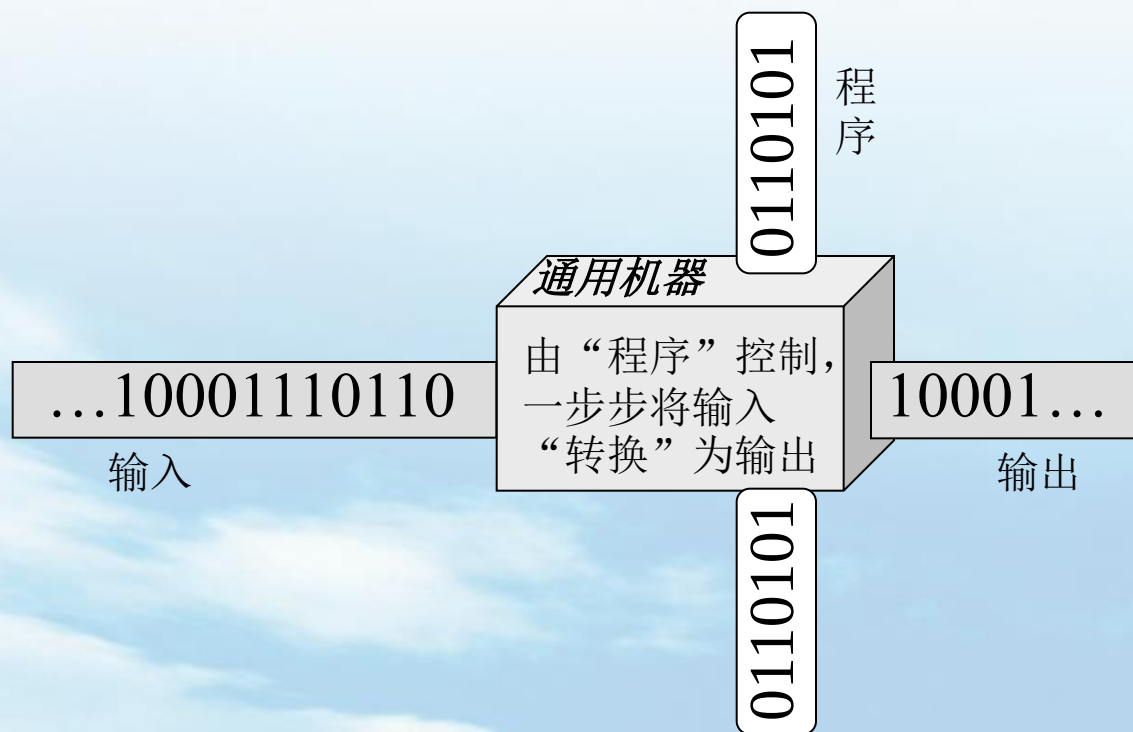
查阅一下哪些人获得了图灵奖？因为什么贡献而获奖呢？





# 图灵认为什么是计算

◆ 所谓**计算**就是计算者(人或机器)对一条两端**可无限延长**的纸带上的一串**0或1**，执行指令一步一步地**改变**纸带上的**0或1**，经过有**限步骤**最后得到一个**满足预先规定的符号串的变换过程**。





# 图灵机的思想

## 关于数据、指令、程序及程序/指令自动执行的基本思想。

- ◆ 输入被制成一串0和1的纸带，送入机器中----**数据**。如00010000100011...
- ◆ 机器可对输入纸带执行的**基本动作**包括：“翻转0为1”，或“翻转1为0”，“前移一位”，“停止”。
- ◆ 对基本动作的控制----**指令**，机器是按照指令的控制选择执行哪一个动作，指令也可以用0和1来表示：**01**表示“翻转0为1”（当输入为1时不变），**10**表示“翻转1为0”（当输入0时不变），**11**表示“前移一位”，**00**表示“停止”。
- ◆ 输入如何变为输出的控制可以用指令编写一个**程序**来完成，如：  
**011110110111011100...**
- ◆ 机器能够读取程序，按程序中的指令顺序读取指令，读一条指令**执行**一条指令。由此实现**自动计算**。





# 图灵机的组成

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

其中:

$Q$ : 状态的有穷集合

$q_0$ : 开始状态

$F$ : 终止状态集合

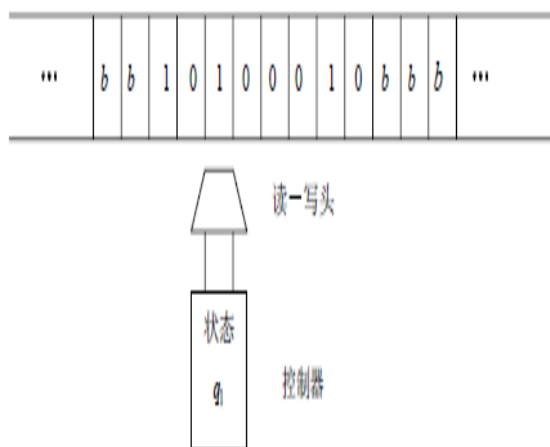
$\Gamma$ : 带符号表

$B$ : 空白符号

$\Sigma$ : 输入字母表

$\delta$ : 移动函数 (1):  $\delta(q, X) = (p, Y, R)$  表示  $M$  在状态  $q$  读入符号  $X$ , 将状态改为  $p$ , 并在这个  $X$  所在的带方格中印刷符号  $Y$ , 然后将读头向右移动一格.

(2):  $\delta(q, X) = (p, Y, L)$  表示  $M$  在状态  $q$  读入符号  $X$ , 将状态改为  $p$ , 并在这个  $X$  所在的带方格中印刷符号  $Y$ , 然后将读头向左移动一格.



## ■ 一条存储带

- 双向无限延长
- 分为一系列均匀的方格
- 每个方格可存储一个数字/字母

## ■ 一个控制器

- 包含一个读写头可以读、写, 更改存储带上每一格的数字/字母
- 可以接受设定好的程序语句
- 可以存储当前自身的状态
- 可以变换自身状态
- 可以沿存储带一格一格地左/右移动





# 程 序

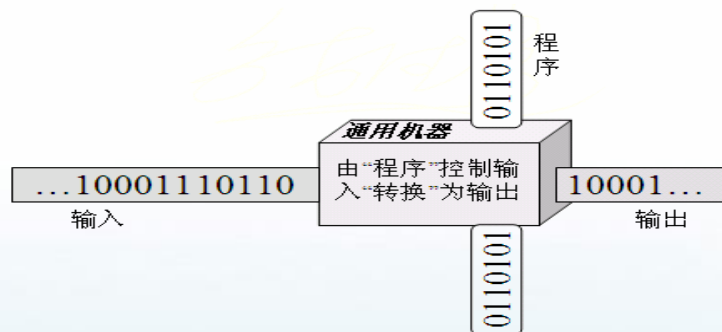
- **程序**是一个五元组 $\langle q, X, Y, R(\text{或} L, N), p \rangle$ 形式的指令集。
  - $q$ 表示机器**当前所处的状态**
  - $X$ 表示机器从方格中**读入的符号**
  - $Y$ 表示机器用来代替 $X$ ，**写入方格中的符号**
  - $R$ 、 $L$ 、 $N$ 分别表示向**右移**一格，向**左移**一格，**不移动**
  - $p$ 表示**下一步机器的状态**



# 图灵机工作原理

## 1. 准备

- 1) 存储带上符号**初始化**
- 2) 控制器设置好自身**当前状态**
- 3) 控制器置于**起始位置**
- 4) 准备好**工作程序**



## 2. 反复执行以下工作直到停机

- 1) 读写头**读出**存储带上当前方格中的字符
- 2) 根据当前位置和读到的字符**找到**相应的程序语句
- 3) 根据相应程序语句**做**三个动作
  - ① 在当前存储带方格上写入一个相应的字符
  - ② 变换自身状态到新状态
  - ③ 读写头向左或向右移一格



# 图灵机的工作原理

例示:

$S_1$ : 开始状态  
 $S_2$ : 右移状态  
 $S_3$ : 左移状态  
 $S_4$ : 停机状态

控制器

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|

|       |   |   |   |       |
|-------|---|---|---|-------|
| $S_1$ | 0 | 0 | R | $S_1$ |
|-------|---|---|---|-------|

$(S_1, 0, 0, R, S_1)$   
 $(S_1, 1, 1, R, S_2)$   
 $(S_2, 1, 1, R, S_2)$   
 $(S_2, 0, 1, L, S_3)$   
 $(S_3, 1, 1, L, S_3)$   
 $(S_3, 0, 0, N, S_4)$

当前机器状态

当前读入的符号

当前应写入的符号

读写头的动作

机器应转入的状态

功能: 将一串连续1的后面再加一位1

$(S_1, 0, 0, R, S_1)$

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|

$(S_1, 1, 1, R, S_2)$

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|

$(S_2, 1, 1, R, S_2)$

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|

$(S_2, 0, 1, L, S_3)$

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|

$(S_3, 1, 1, L, S_3)$

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|

$(S_3, 0, 0, N, S_4)$

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|

执行过程



# 图灵机的特点

- 简单
  - 图灵机一定程度上反映了人们最初等、最原始的计算能力，它的基本动作非常简单、机械、确定
- 强大
  - 凡是能用算法方法解决的问题也一定能用图灵机解决；凡是图灵机解决不了的问题任何算法也解决不了——图灵可计算性问题
- 可实现
  - 并非只停留在模型阶段，图灵机是切实可模拟实现的

# 图灵机的特点

- 简单

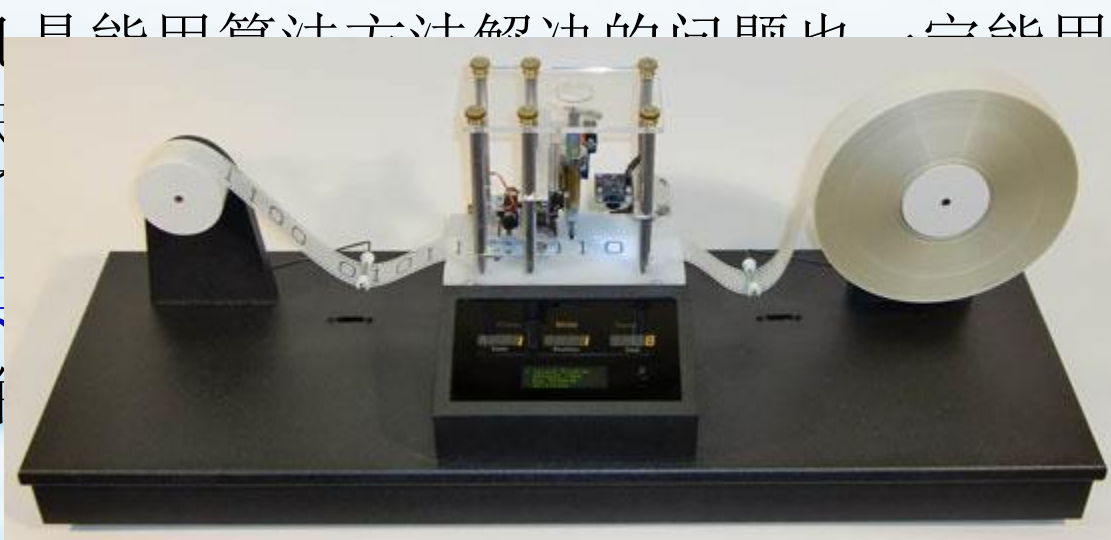
- 图灵机一定程度上反映了人们最初等、最原始的计算能力，它的基本动作非常简单、机械、确定

- 强大

- 凡能用图灵机解决的问题，图灵机也能解决

- 可实现

- 并能实现



<http://aturingmachine.com/>





# 图灵机的意义

◆ **图灵机模型**被认为是计算机的基本理论模型。

——计算机是使用相应的程序来完成任何设定好的任务。

图灵机是一种离散的、有穷的、**构造性的**问题求解思路，  
**一个问题的求解可以通过构造其图灵机(即程序)来解决。**

◆ 解决了可计算性的判定问题

——**凡是能用算法方法解决的问题也一定能用图灵机解决；**  
**凡是图灵机解决不了的问题任何算法也解决不了**----**丘奇-**

**图灵论题**



# 小结



输入/输出都是0和1的形式表达

程序和指令也是0和1的形式表达

程序可用状态转换图来表达



## 进一步阅读文献

- 《计算机科学概论》（人民邮电出版社）  
第12.1-12.2节（中/英）
- 《计算思维导论》2.3.1节

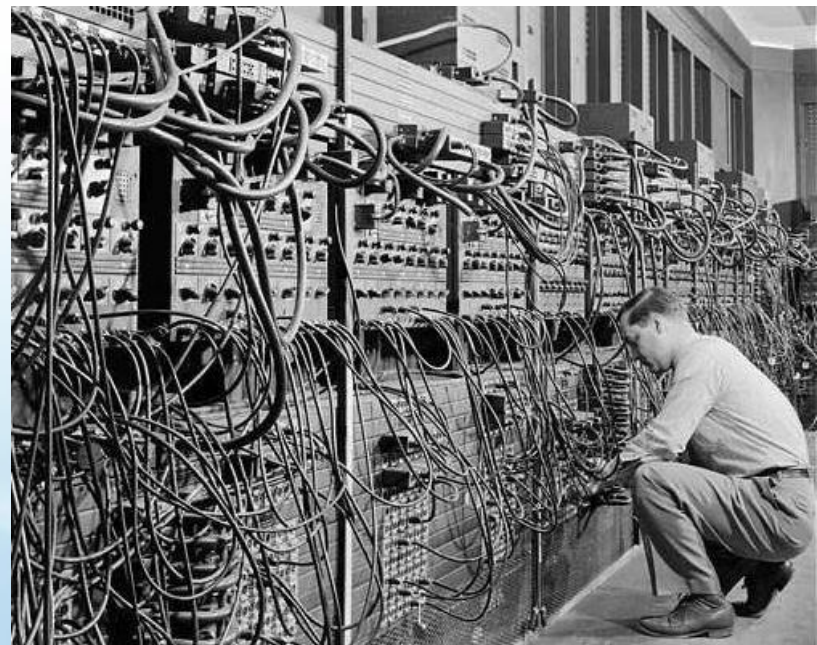


## 目标三

# 冯·诺依曼计算机：思想与构成

# 引入

- 已知
  - 电路是能完成计算的, 所以, 可以设计 加法、减法、乘法... 各种电路..., 用它们来完成各种类型的计算。
- 那么...
  - 需要完成什么计算, 就动手设计一个相应的电路
  - 设计好很多个原子电路, 需要的时候我就把它们临时组装在一起
  - 这就是 ENIAC 的原理!





# 引入

- 所以：
  - 不能通过重新“组合不同电路”的方式，去完成的计算任务！
- 而应该：
  - 通过某种命令来控制计算机，让计算机按照这种命令来运行，这种命令可以用电信号表示；
  - 这种命令不是“临时输入”到计算机，而是存放在某个地方，随时可以更改；
  - 命令改了，计算机的功能随之改变；
  - 这就是EDVAC！



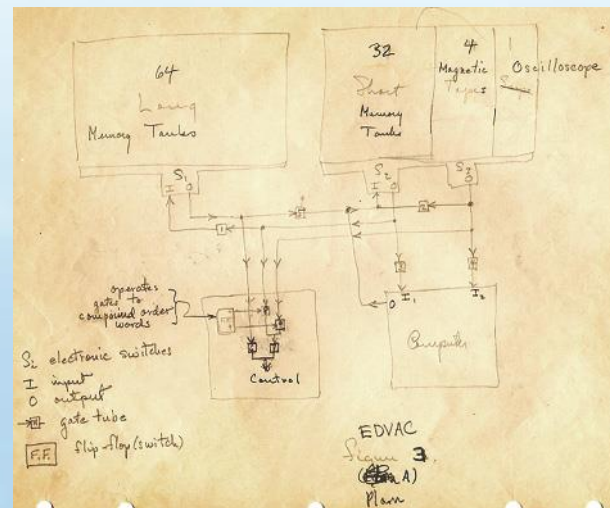
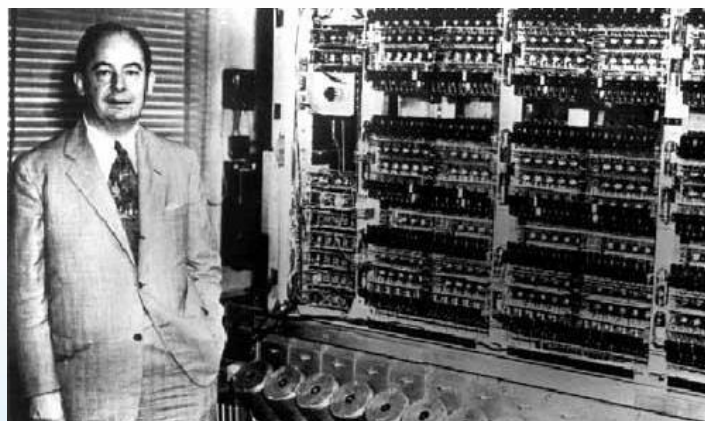
冯·诺依曼(John von Neumann)

☞ 美籍匈牙利人

☞ 1903–1957

# 什么是冯·诺依曼计算机？

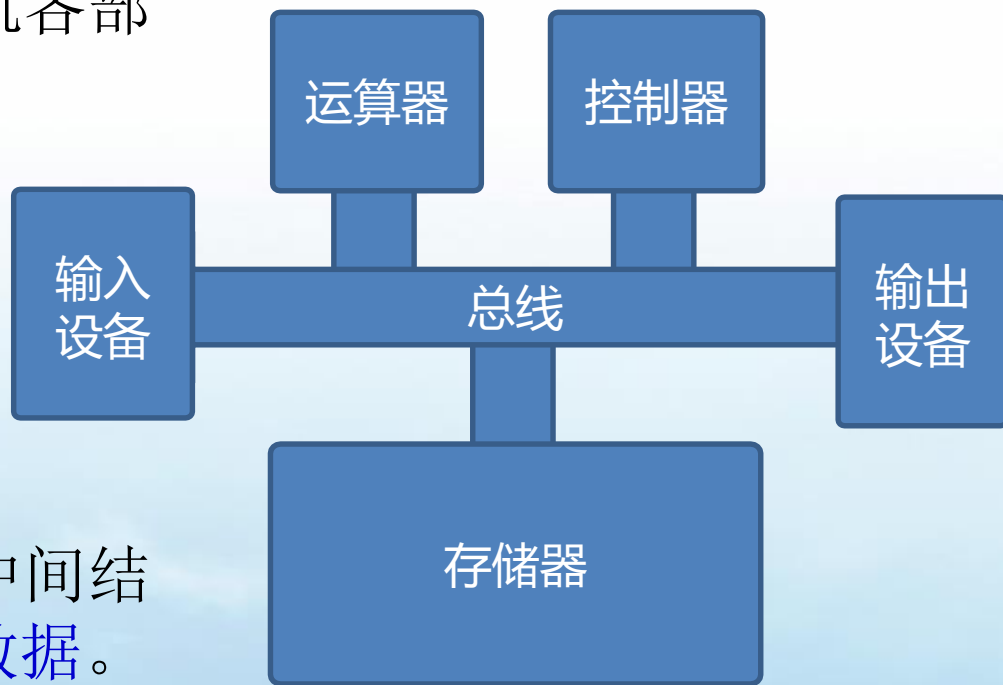
- ◆ 1945年6月，冯·诺伊曼以“关于EDVAC的报告草案”为题，起草了长达101页的总结报告，即计算机史上著名的“101页报告”。
- ◆ 提出“**存储程序**”的计算机设计思想，并进行了实践，现代计算机普遍来讲属于冯·诺伊曼机体系。
- ◆ 冯·诺依曼被称为“**现代计算机之父**”。





# 冯·诺伊曼机的基本思想

- 控制器
  - 统一指挥并控制计算机各部件协调工作
- 运算器
  - 对数据进行算术运算和逻辑运算。
- 存储器
  - 存储待操作的信息与中间结果，包括机器指令和数据。
- 输入设备
- 输出设备

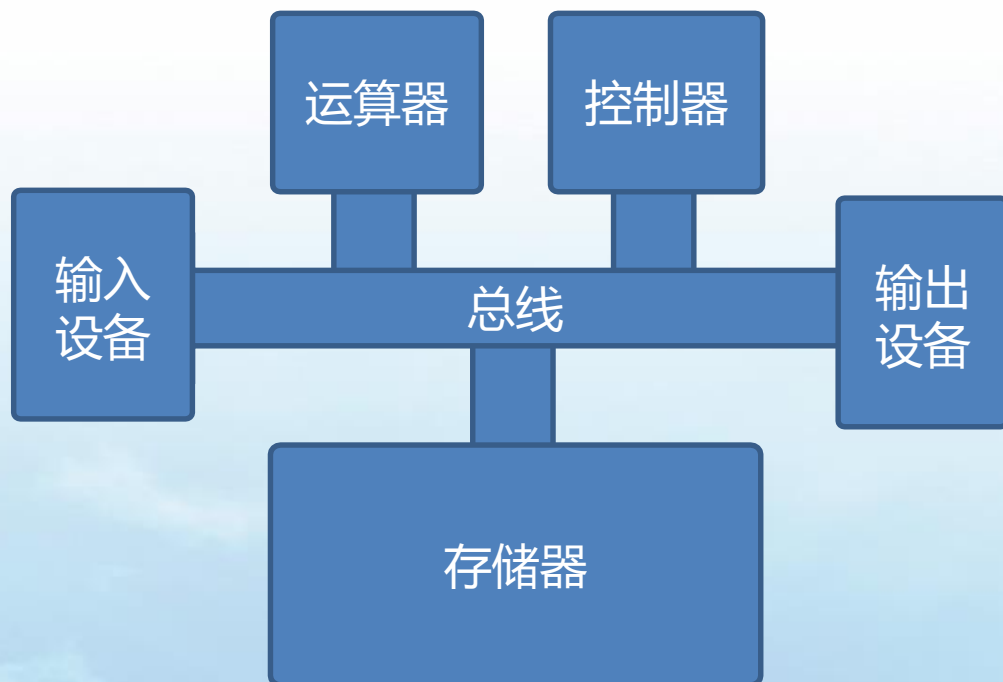




# 冯·诺伊曼机的基本思想

## • 工作过程

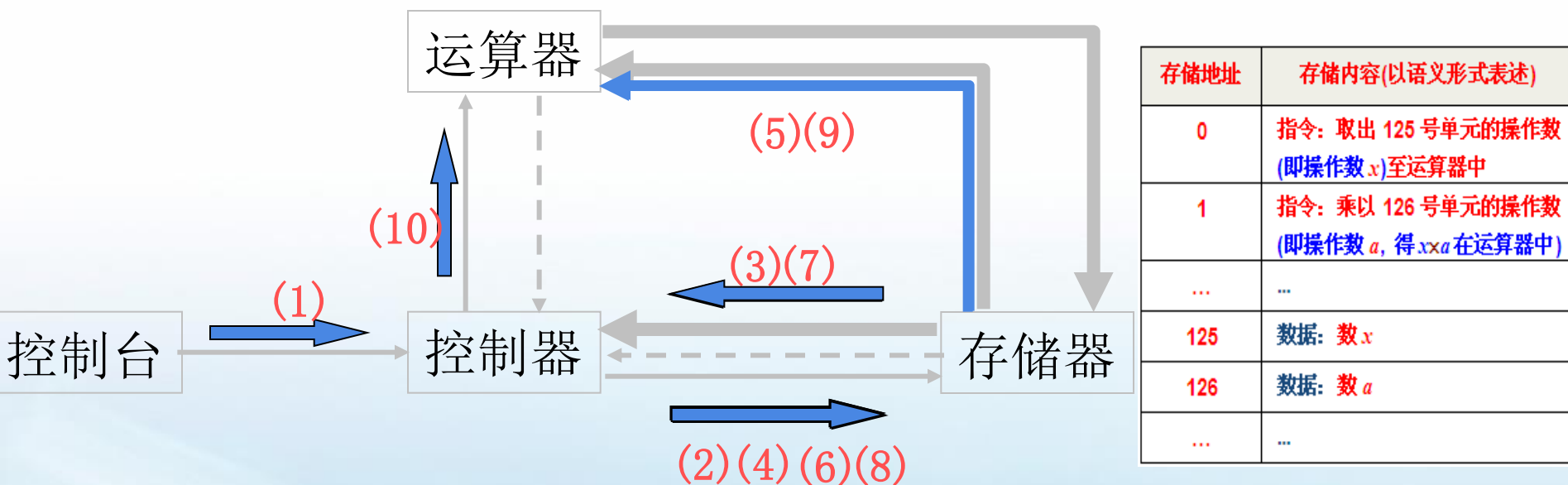
- 在 **控制器** 指挥下，从 **存储器** 上取出指令；
- 分析指令，得到计算命令 和 待操作的数；
- 从 **存储器** 上取出待计算的数 放入 **运算器**
- **运算器** 计算结果
- 输出到 **存储器** 或 **输出设备**







# 冯·诺依曼机工作原理



- (1) 启动控制器工作
- (2) 发送第1条指令地址
- (3) 取出指令并分析指令
- (4) 执行指令: 发送操作数 $x$ 所在地址
- (5) 执行指令: 取出操作数 $x$

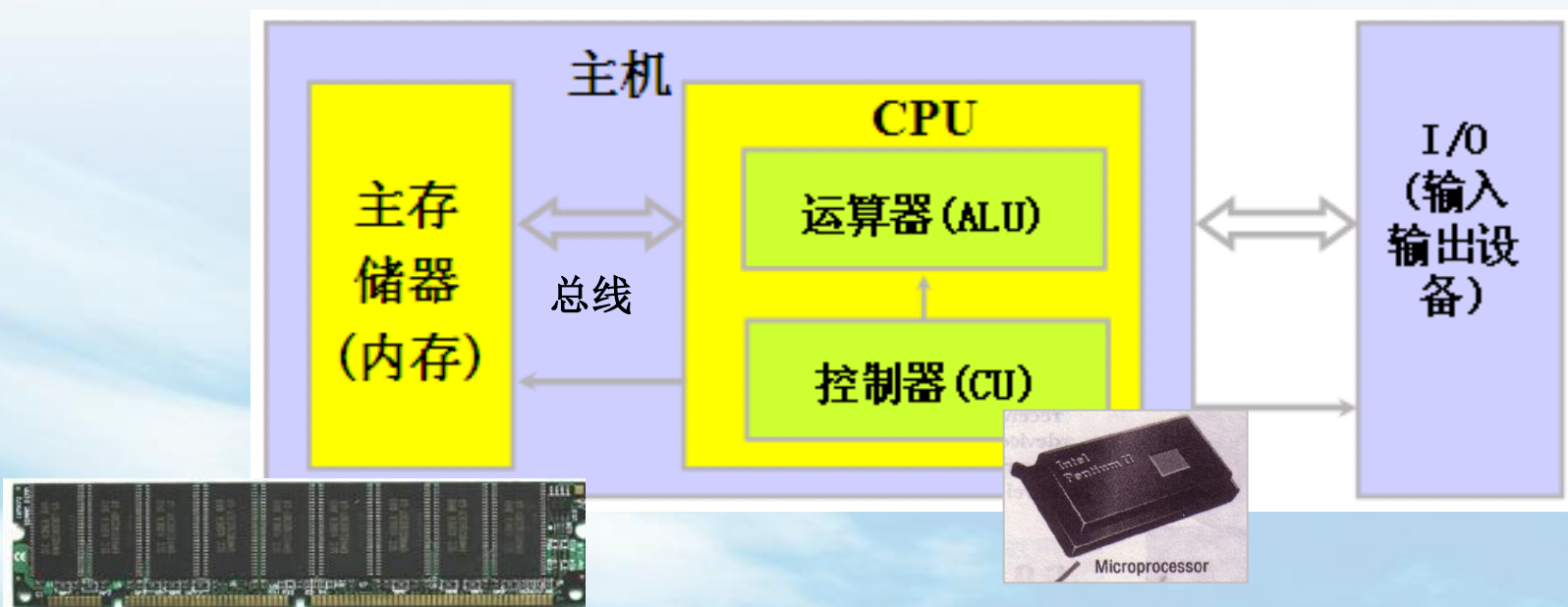
- (6) 发送下一条指令地址
- (7) 取出指令并分析指令
- (8) 执行指令: 发送操作数 $a$ 所在地址
- (9) 执行指令: 取出操作数 $a$
- (10) 执行指令: 通知运算器计算 $a$ 乘 $x$
- (11) 继续后续指令的取指、执行...





# 现代计算机的基本部件

- ◆ **CPU**: 中央处理单元(Central Process Unit), 将运算器和控制器集成在一块芯片上, 形成微处理器。
- ◆ **CPU、主存储器、I/O设备及总线**成为现代计算机的四大核心部件。



现代计算机里面, 一个微处理器(芯片)可能包含多个**CPU**, 即多核.



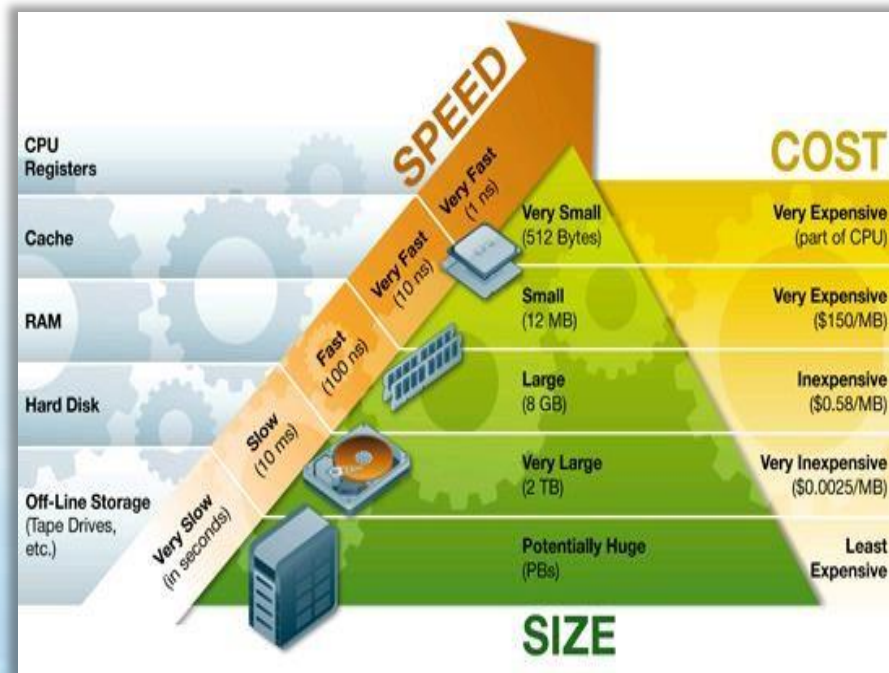
# 输入设备和输出设备

- 作用：将信息输入计算机和输出计算机。
- 常用的文字输入设备是键盘（还有扫描仪、穿孔卡片读入机和鼠标等专用输入设备）。
  - 当在键盘上按下一个键时，按下的键通过编码变换成机器可读的数据形式，
  - 如字符“A”变换成ASCII码“1000001”，该编码数据随即存入存储器等待处理，
  - 通过与“1000001”对应的字符点阵数据在屏幕上显示一个字符“A”。
- 输出设备有打印机、显示器、绘图仪、磁记录设备等。



# 存储器

- 寄存器
  - CPU内部，用于存放待操作数和结果
- 高速缓存
  - 通常在CPU内部，用做数据缓冲区
- 内存
  - CPU里想放但放不下的部分
- 外存

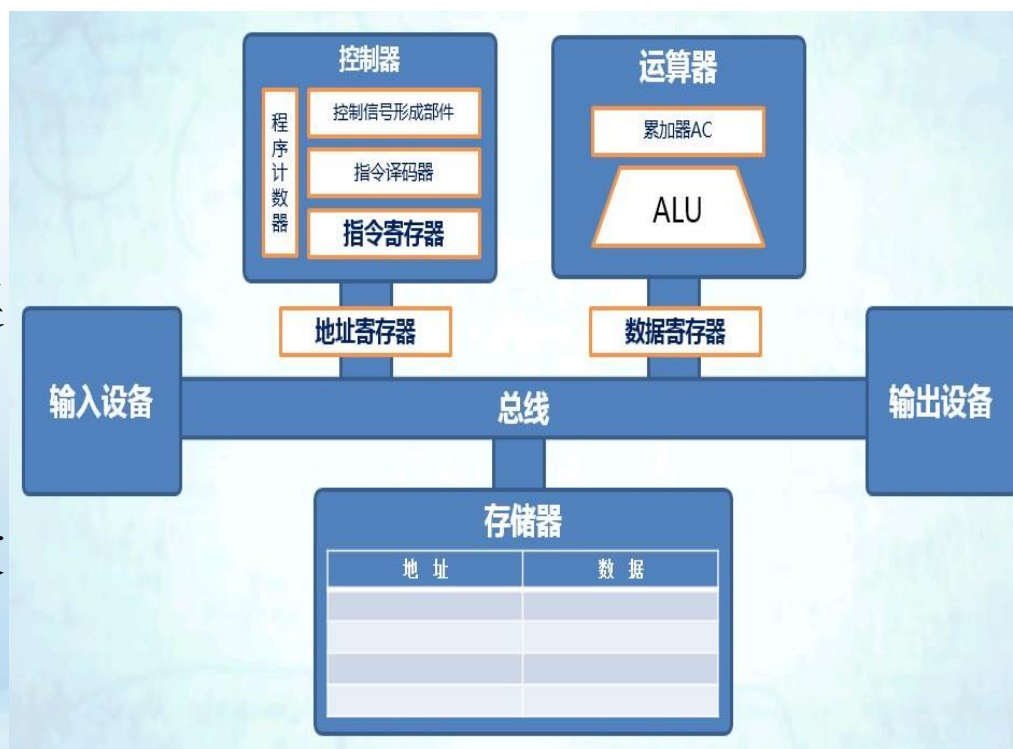




# 存储器

- 寄存器

- 工作速度与CPU运算部件频率一致
- 一次存取数据大约花费0.x纳秒的时间
- 寄存器与运算部件直接连接，运算部件直接对寄存器进行读写操作
- 寄存器制作成本高，一般CPU芯片中通常只配备少数寄存器

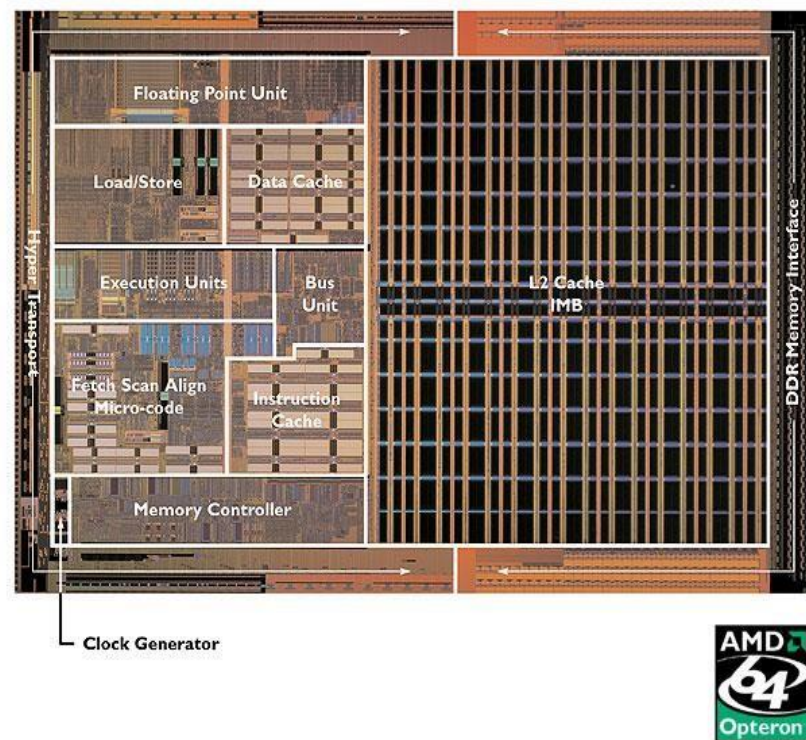






# 存储器

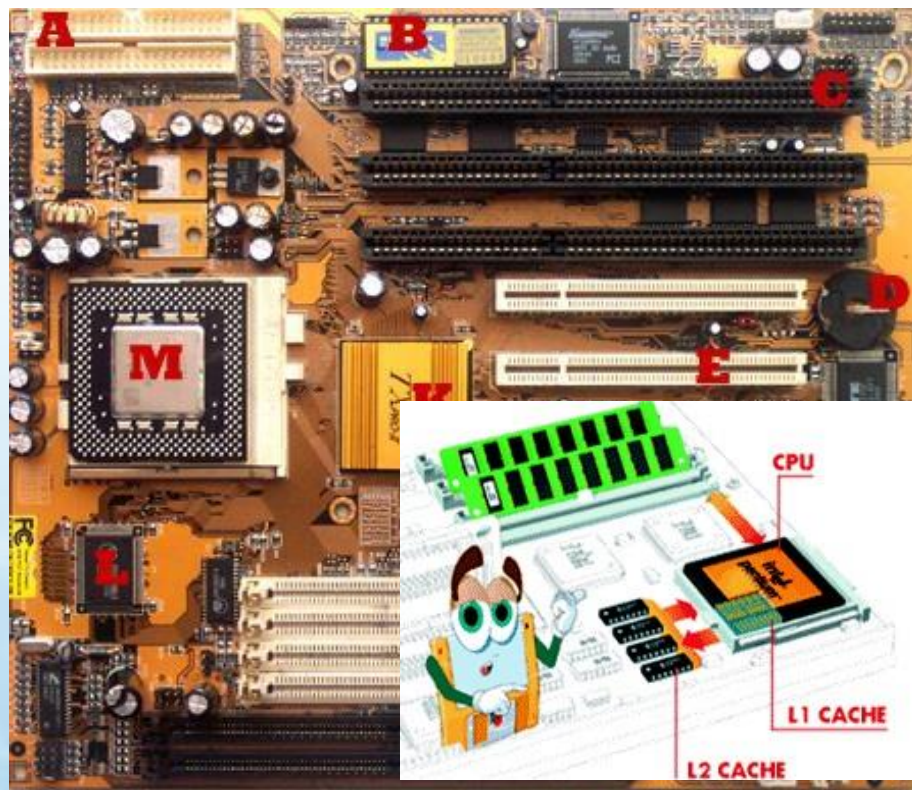
- CACHE (高速缓存)
  - CPU内部的缓存叫内部高速缓存 (Internal Cache)
  - 主板上的缓存叫外部高速缓存 (External Cache)
  - 通常又分为多级，不同级之间工作频率不同
  - 一般在纳秒内读写一次
  - 容量大小不同 (nK-nM)





# 存储器

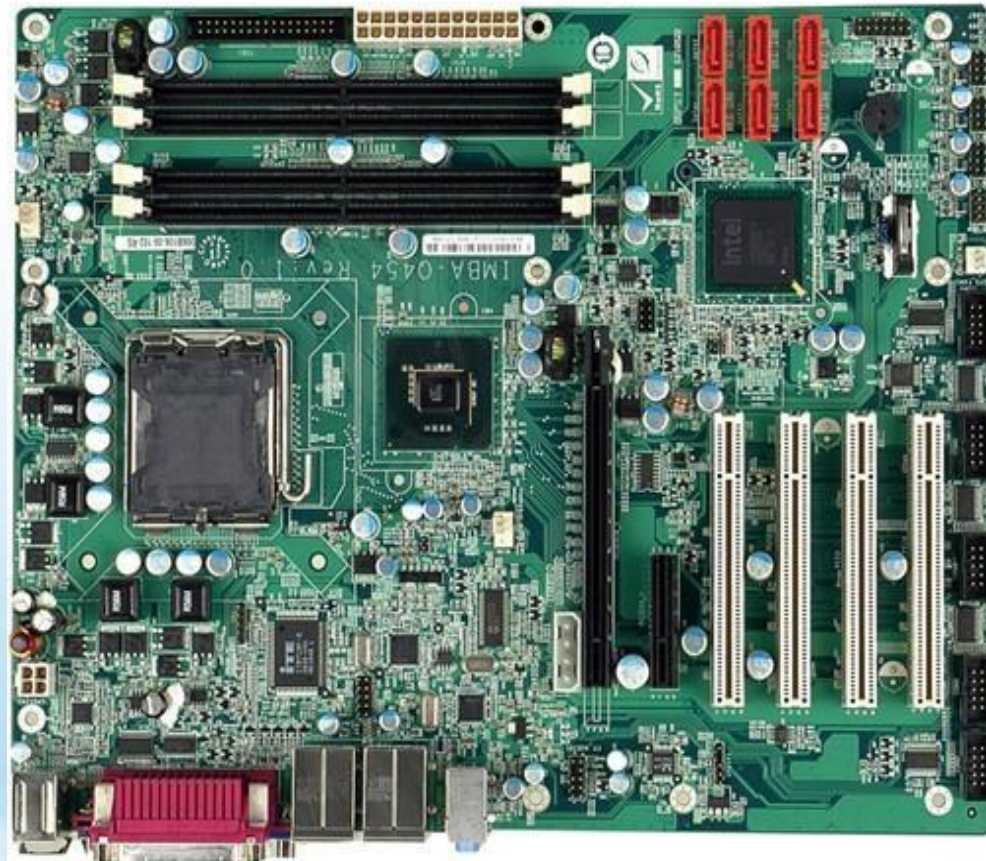
- CACHE (高速缓存)
  - CPU内部的缓存叫内部高速缓存 (Internal Cache)
  - 主板上的缓存叫外部高速缓存 (External Cache)
  - 通常又分为多级, 不同级之间工作频率不同
  - 一般在纳秒内读写一次
  - 容量大小不同 (nK-nM)



# 存储器

- 主存储器

- 内存储器，简称内存
- 存放CPU中的运算数据
- 存放与硬盘等外部存储器交换的数据
- 临时存放，断电丢失
- 价格低、易于更换
- 工作频率低于CPU工作频率，但仍有差距





# 存储器

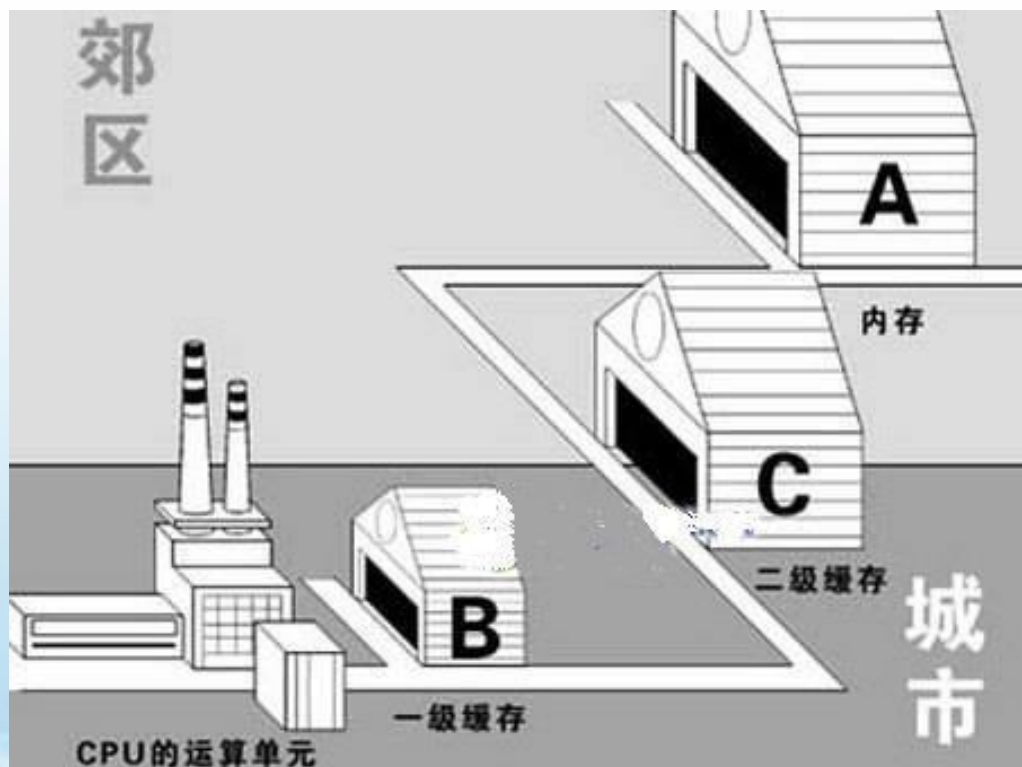
## • 外存储器

- 外存储器是指除计算机内存及CPU缓存以外的存储器
- 存放数据、资料等，可供内存读取
- 长期存放，不依赖通电
- 价格更低、易于更换
- 工作频率较低



# 存储器

- CPU读取数据时
  - 先从缓存中查找，找到立即读取；
  - 找不到，就用相对慢的速率从内存中读取并送给CPU处理
  - 同时把这个数据所在的数据块调入缓存中
  - 可以使得以后对整块数据的读取都从缓存中进行，不必再调用内存。
- 尽量减少可能的“慢”的读取操作

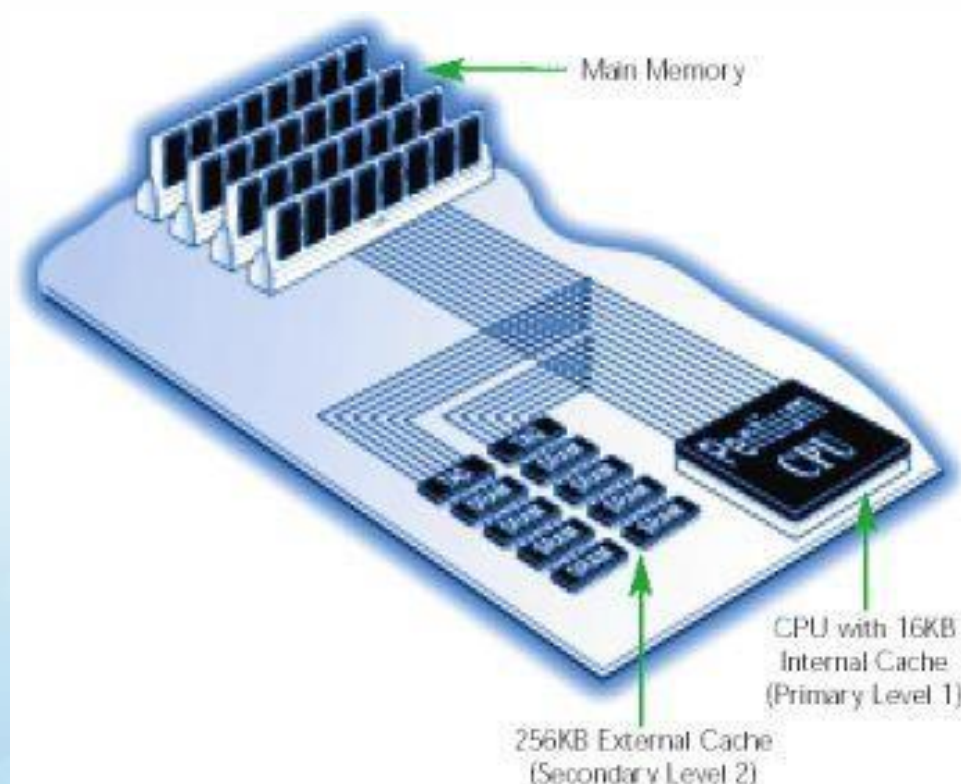


<http://baike.baidu.com/view/907.html>

# 存储器

## 局部性原理

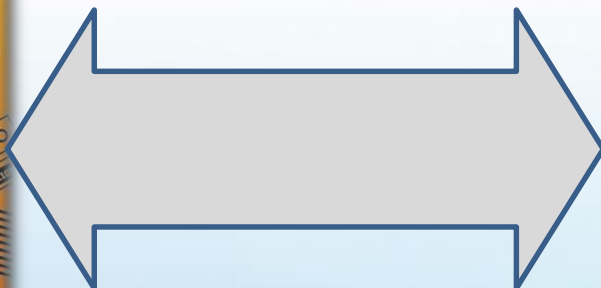
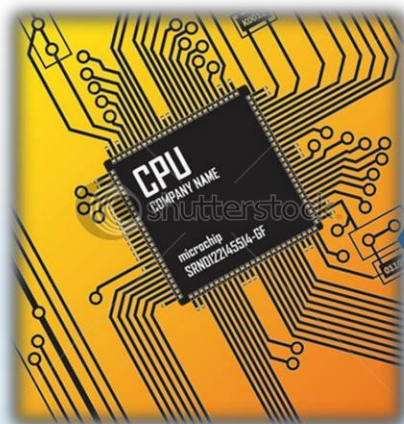
- CPU对数据的访问通常具有一定的局部性:
- 时间局部性
  - 如果一个内存地址正在被使用, 那么在近期他很可能还会再次被访问
- 空间局部性
  - 在最近的将来可能用到的信息很可能是与当前使用的信息相邻的







# CPU位置与寻址空间



0x0012FF70

0x0012FF72

0x0012FF74

0x0012FF76

0x0012FF78

0x0012FF7A

内存地址

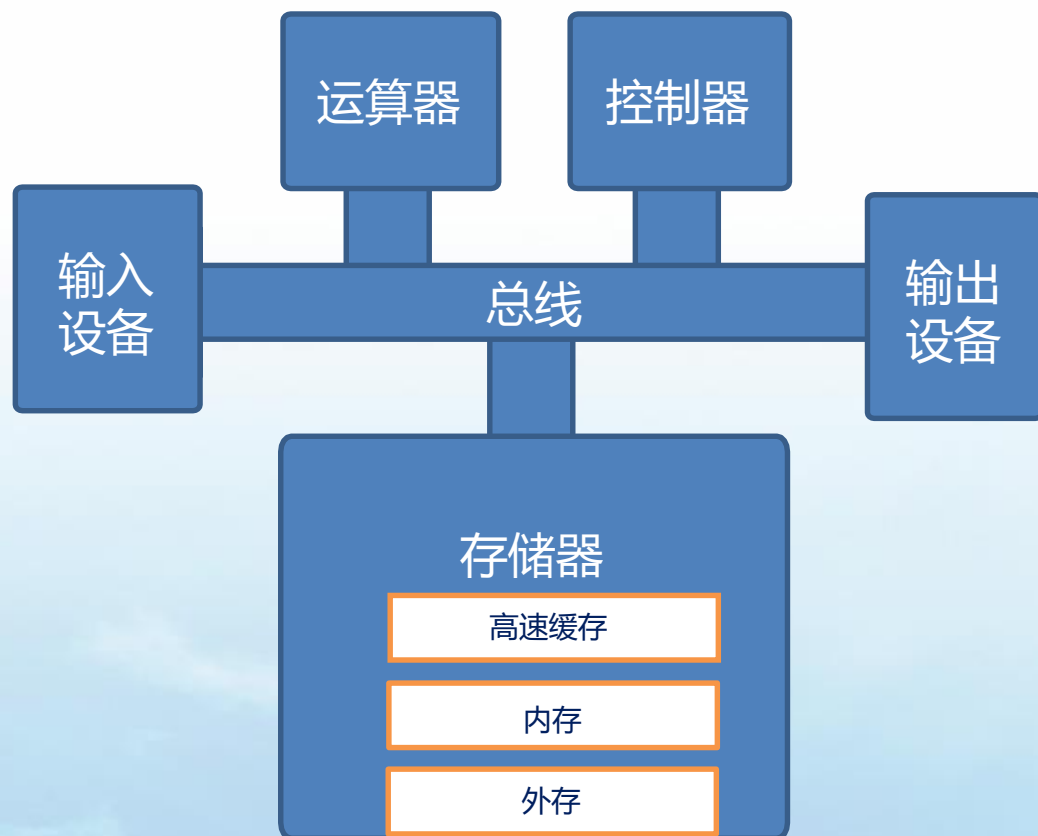
|     |
|-----|
| ... |
| 15  |
| 2   |
| 76  |
| 30  |
| 126 |
| 5   |
| ... |

为什么32位的CPU最多只能配备4G内存？



# 冯·诺伊曼式机命令的执行

- 已知 CPU电路能进行计算
- 冯诺依曼计算机
  - 不需要通过“插线”临时组合电路完成不同任务
  - 可以通过读取并执行“存储好的”命令来完成不同功能





# CPU的指令集

- 指令集

- CPU中用来计算和控制计算机系统的一套指令的集合
- 在CPU设计时就预先定义好的
- 体现CPU性能的重要标志。
- Intel X86指令集
- ARM指令集

## Intel X86基本指令集

### 目录 [隐藏]

- 1 资料搬移指令 (Data transfer instructions)
- 2 整数运算指令
  - 2.1 加法
  - 2.2 减法
  - 2.3 乘法
  - 2.4 除法
- 3 逻辑运算指令
- 4 位元位移指令
- 5 程式流程控制
  - 5.1 跳越指令
    - 5.1.1 无条件跳越
    - 5.1.2 无号数运算(比较)后使用的条件跳越指令
    - 5.1.3 有号数运算(比较)后使用的条件跳越指令
    - 5.1.4 依旗号状态判断的条件跳越指令
  - 5.2 循环控制指令
  - 5.3 副程式指令
  - 5.4 中断指令 (Interrupt Instruction)
- 6 旗标控制相关指令
  - 6.1 中断标志位
  - 6.2 进位标志位
- 7 字串处理相关指令
- 8 无运算指令



# CPU能接受的指令

## • 指令

- 最终表现为二进制码
- 其长度随CPU类型不同而不同;
- 包含一个或多个字节
- 包含 指令码 和 操作数
  - 指令码 说明要做的动作
  - 操作数 是指要操作的数或地址

|                  |                   |
|------------------|-------------------|
| 1001010101010011 | 00011110          |
| 1100011101010001 | 01111011 10100001 |
| 1001010001110111 | 00111001 10000001 |
| 1100010101000101 | 01100111          |
| 1001010101010011 | 00101010 10000111 |

### • 零地址指令

|    |
|----|
| OP |
|----|

- 例如: **NOP**、**HLT**, 也叫无操作数指令

### • 一地址指令

|    |   |
|----|---|
| OP | A |
|----|---|

- 例如: 递增, 移位, 取反, **INC AX**, **NOT BX**

### • 二地址指令

|    |    |    |
|----|----|----|
| OP | A1 | A2 |
|----|----|----|

- 例:  $[A1] + [A2] \rightarrow [A1]$ ,  $[A2]$ 为源地址,  $[A1]$ 目的地址

### • 三地址指令

|    |    |    |    |
|----|----|----|----|
| OP | A1 | A2 | A3 |
|----|----|----|----|

- 例:  $[A1] + [A2] \rightarrow [A3]$ , 其中 $[A1]$ 、 $[A2]$ 为源地址,  $[A3]$ 为目的地址, 操作后源地址内容不变仅被拷贝



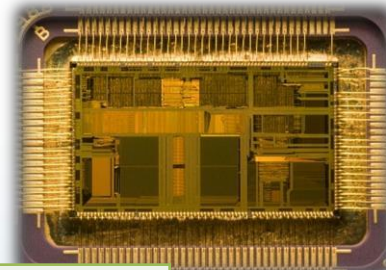


# CPU的指令集

- CPU不是随便什么程序都能执行；
- 它执行规定的指令集中的指令；
- 指令是二进制编码；
- 要把你的命令 转换成满足指令集要求的 二进制代码，才能在计算机上运行；
- 然后才能在CPU上执行

想要计算机  
做的事情

|                  |                   |
|------------------|-------------------|
| 1001010101010011 | 00011110          |
| 1100011101010001 | 01111011 10100001 |
| 1001010001110111 | 00111001 10000001 |
| 1100010101000101 | 01100111          |
| 1001010101010011 | 00101010 10000111 |



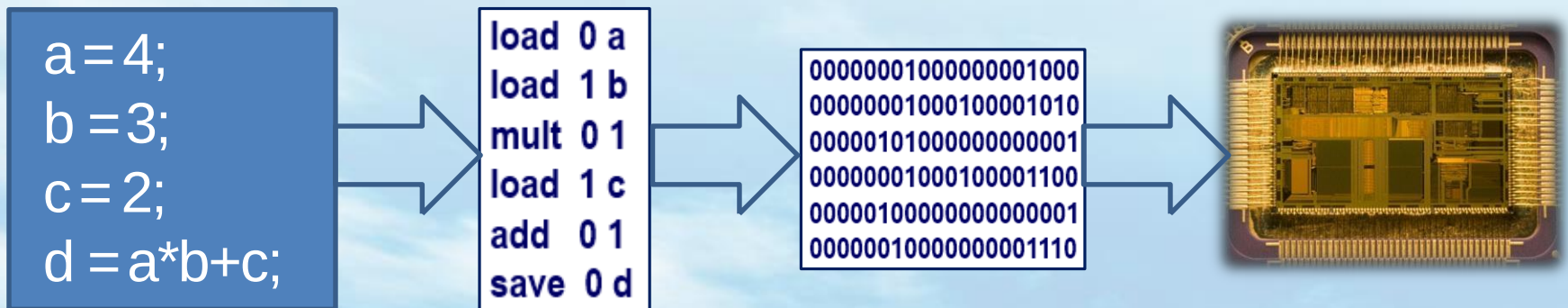


# 程序

- 程序都长这样？

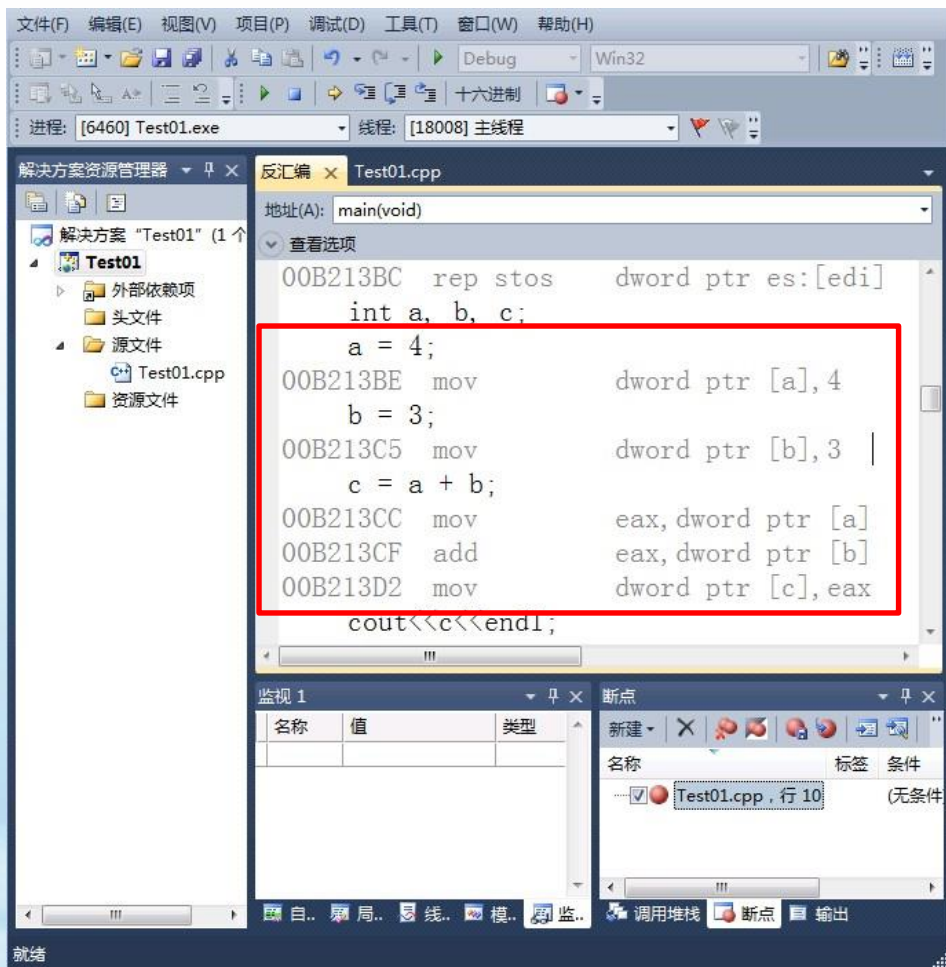
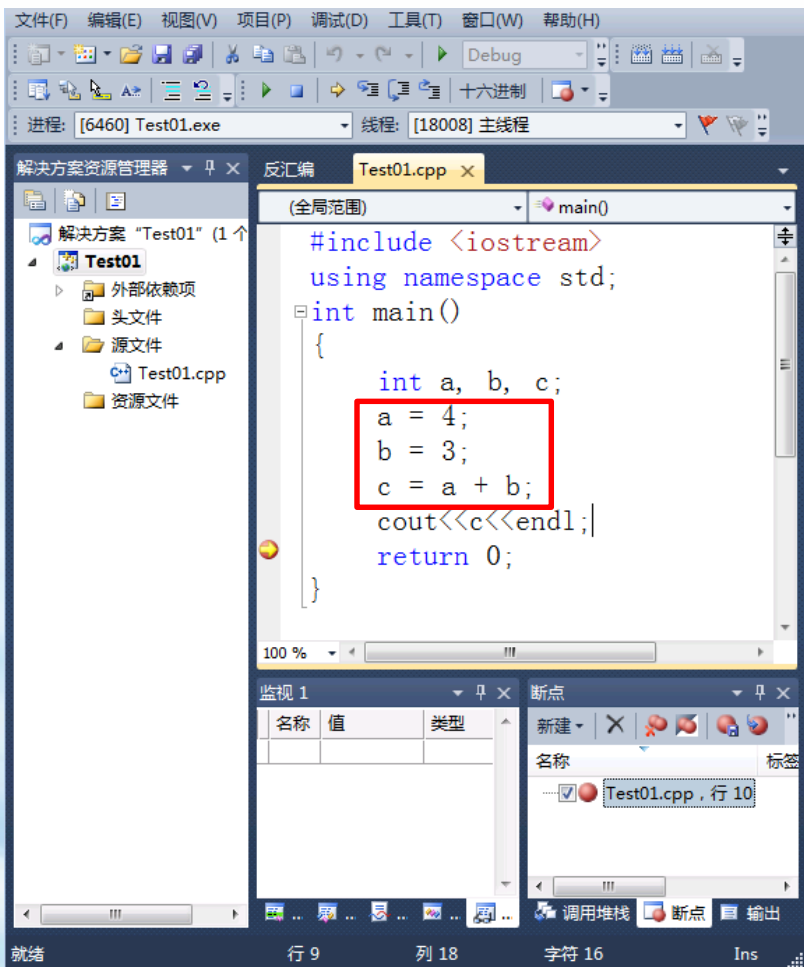
|                  |                   |
|------------------|-------------------|
| 1001010101010011 | 00011110          |
| 1100011101010001 | 01111011 10100001 |
| 1001010001110111 | 00111001 10000001 |
| 1100010101000101 | 01100111          |
| 1001010101010011 | 00101010 10000111 |

- 当然不是！





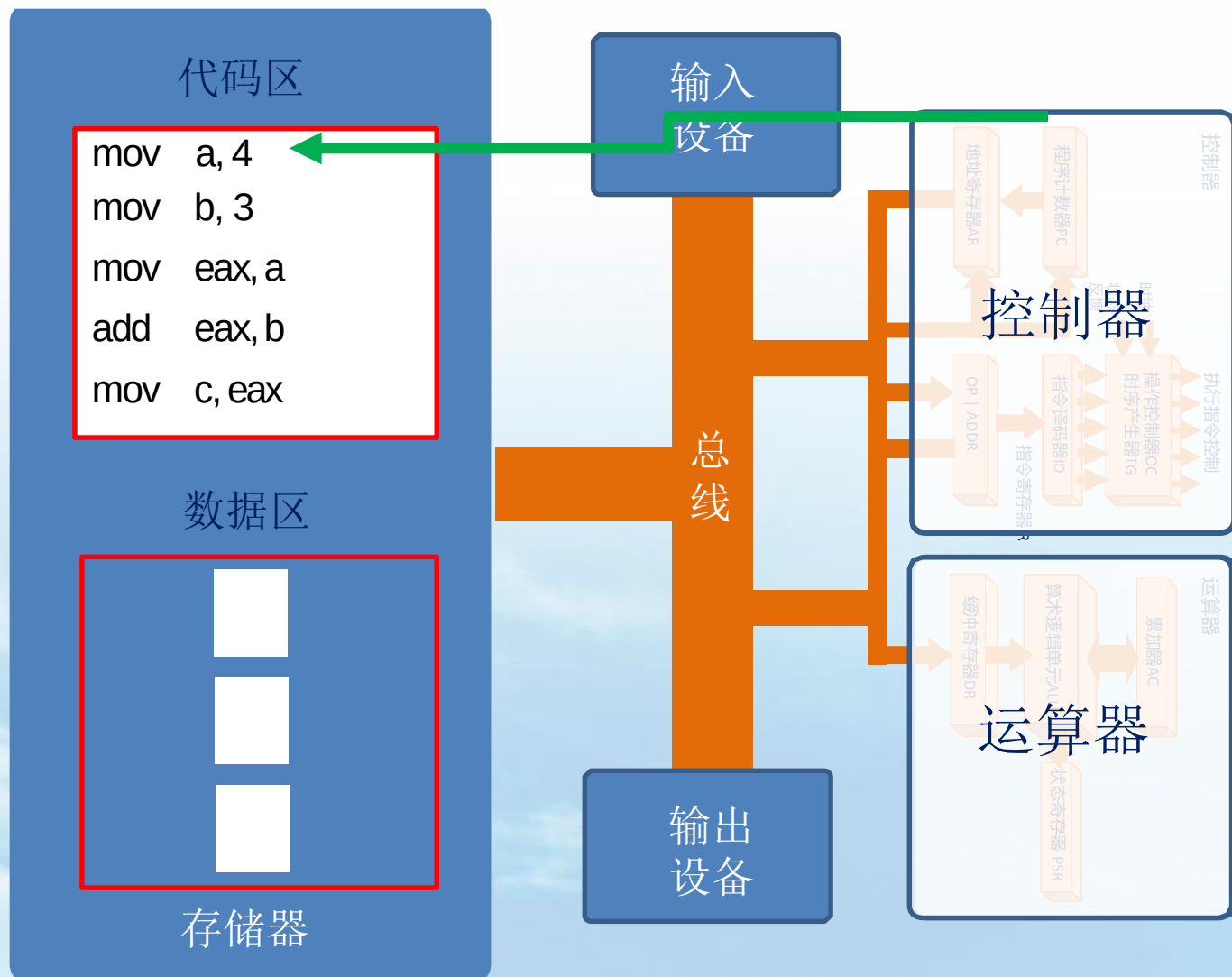
# 程序执行示例





# 程序执行示例

```
a = 4;  
b = 3;  
c = a + b;
```

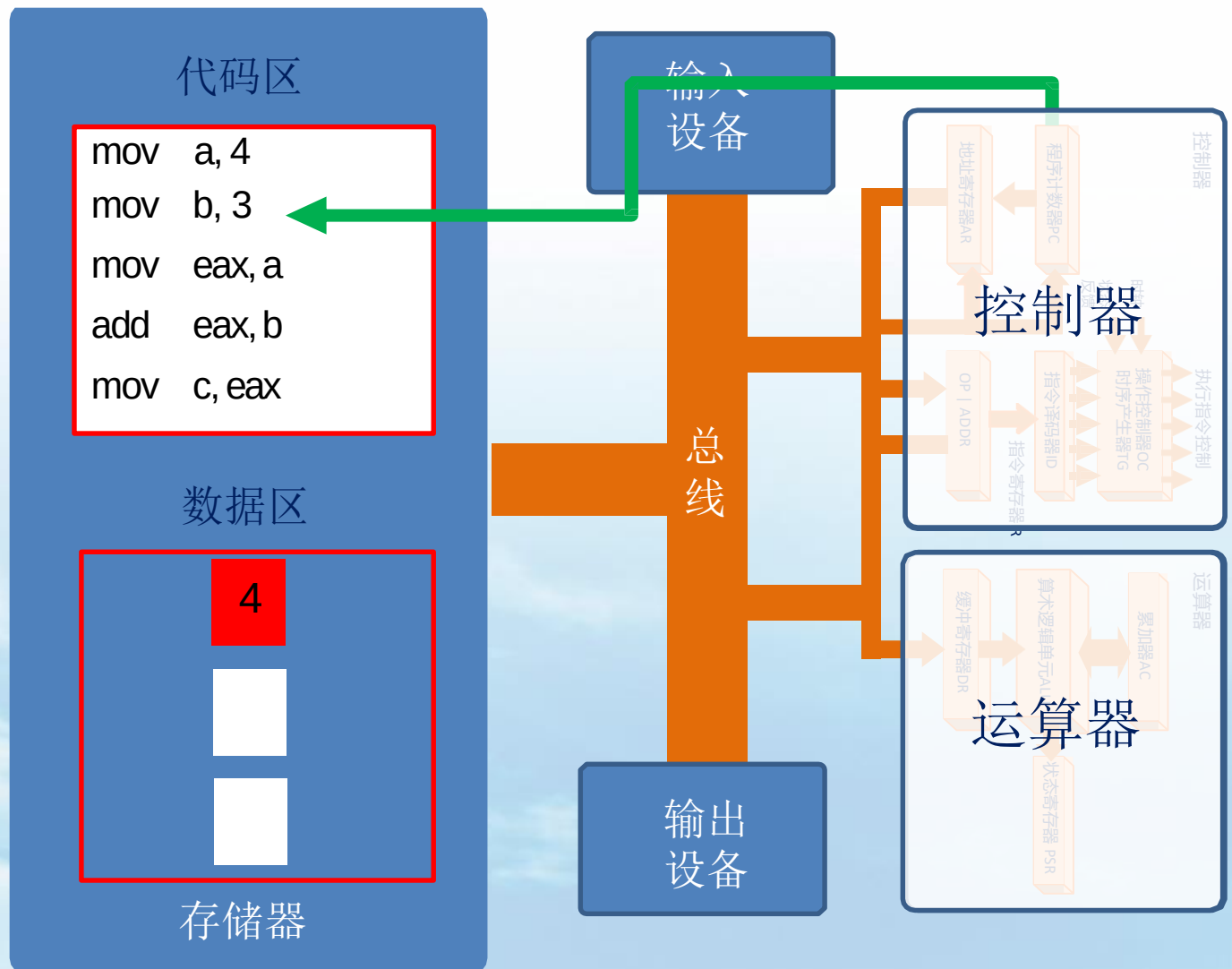






# 程序执行示例

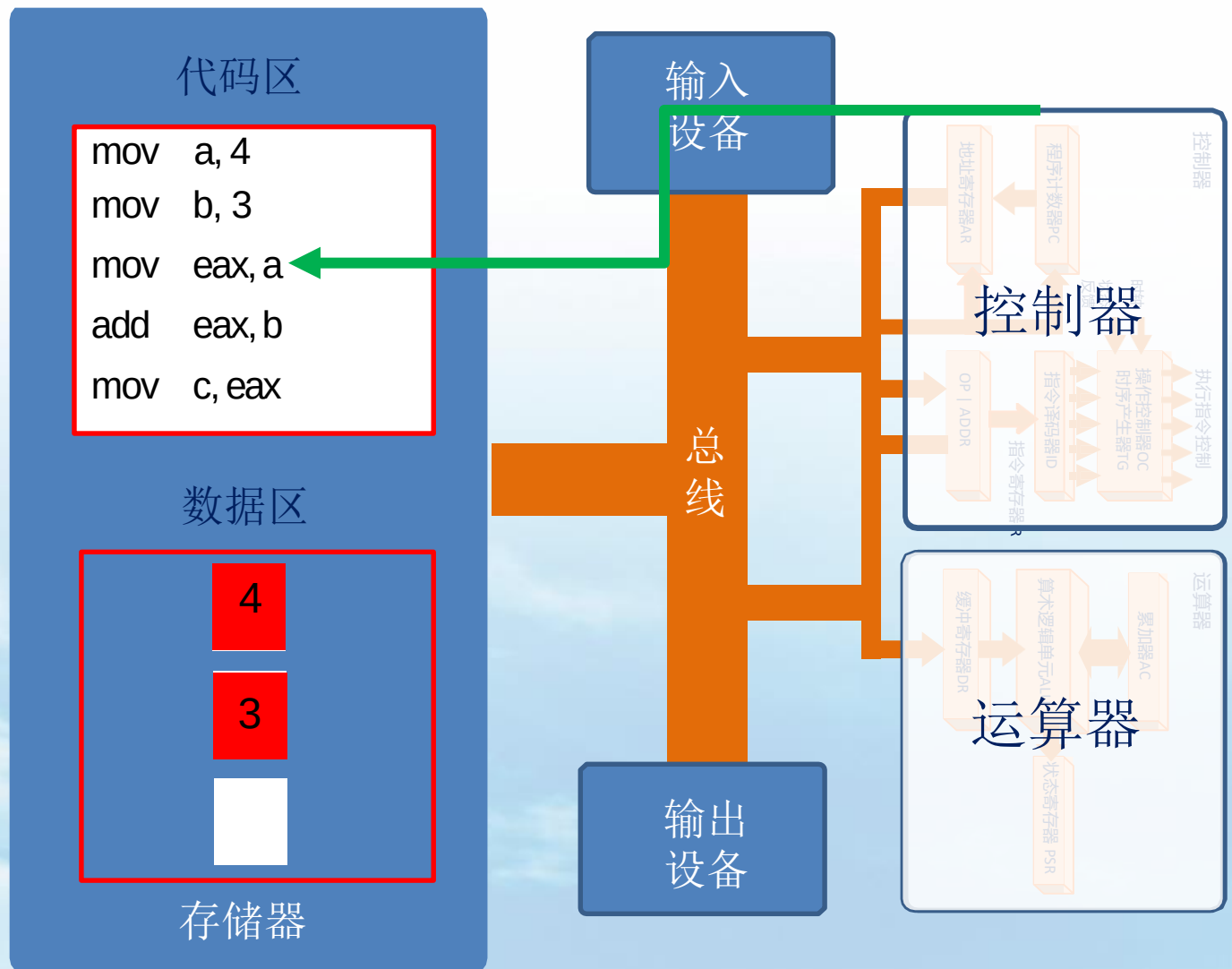
```
a = 4;  
b = 3;  
c = a + b;
```





# 程序执行示例

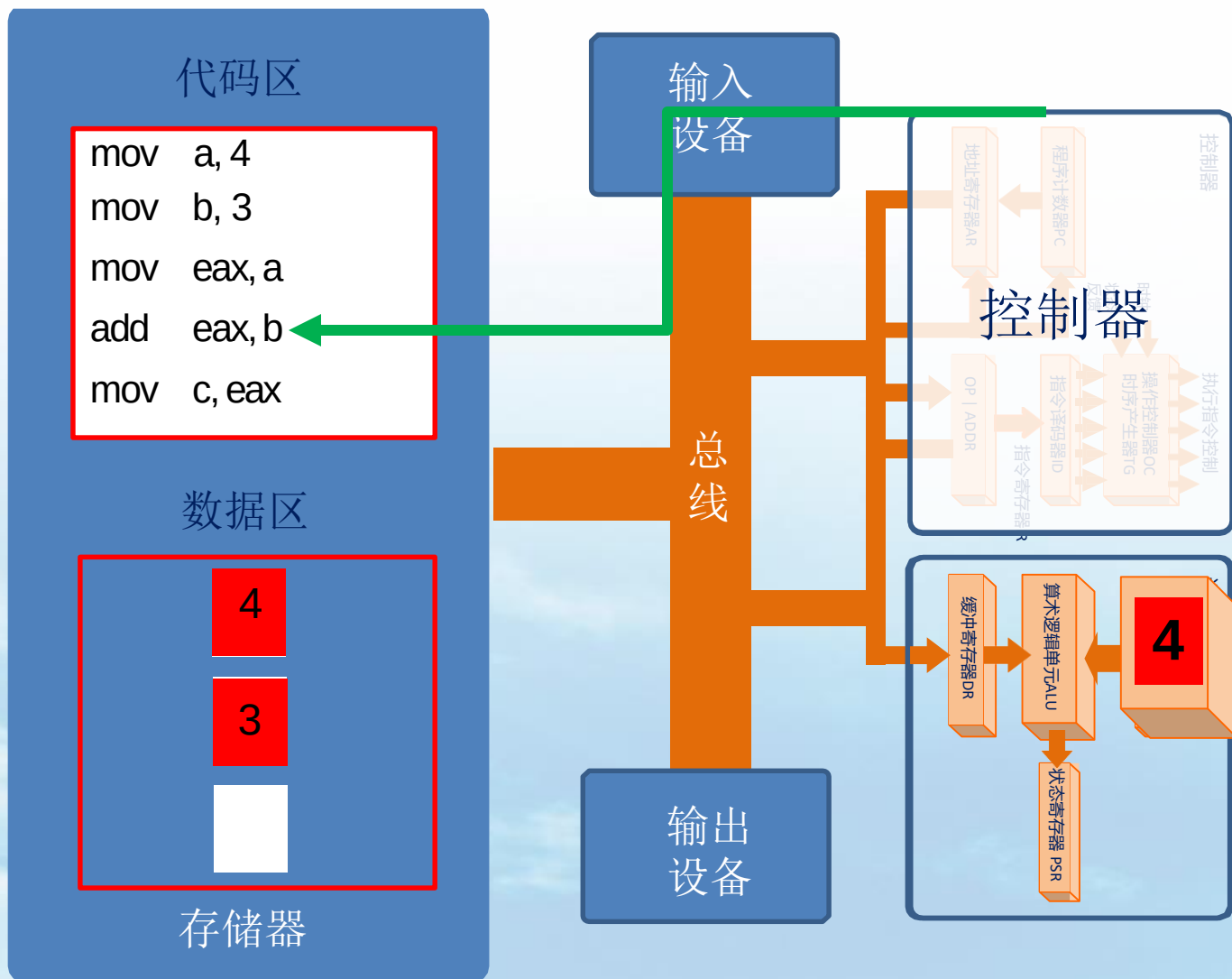
```
a = 4;  
b = 3;  
c = a + b;
```





# 程序执行示例

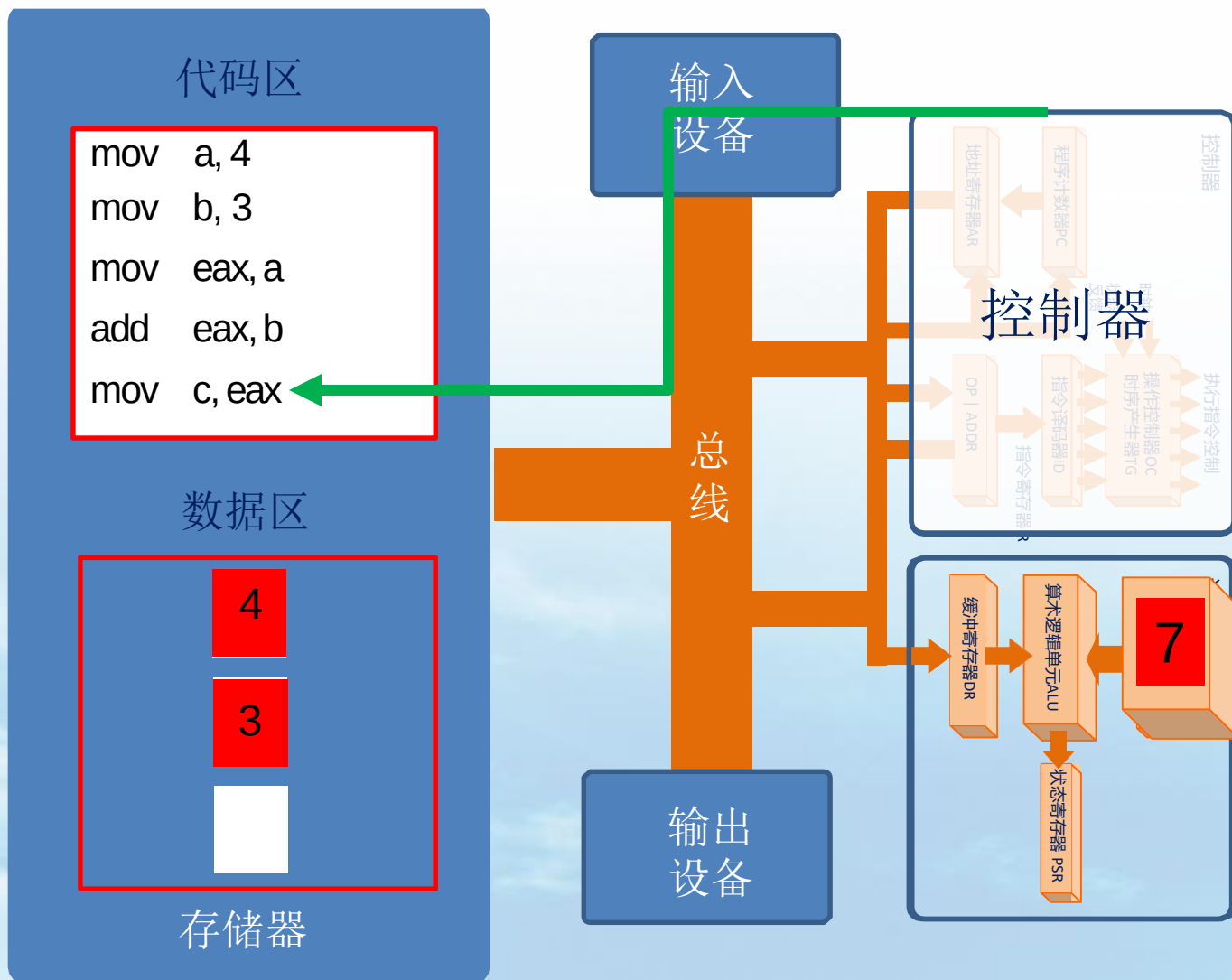
```
a = 4;  
b = 3;  
c = a + b;
```





# 程序执行示例

```
a = 4;  
b = 3;  
c = a + b;
```







```
a = 4;  
b = 3;  
c = a + b;
```

```
mov    a, 4
mov    b, 3
mov    eax, a
add    eax, b
mov    c, eax
```

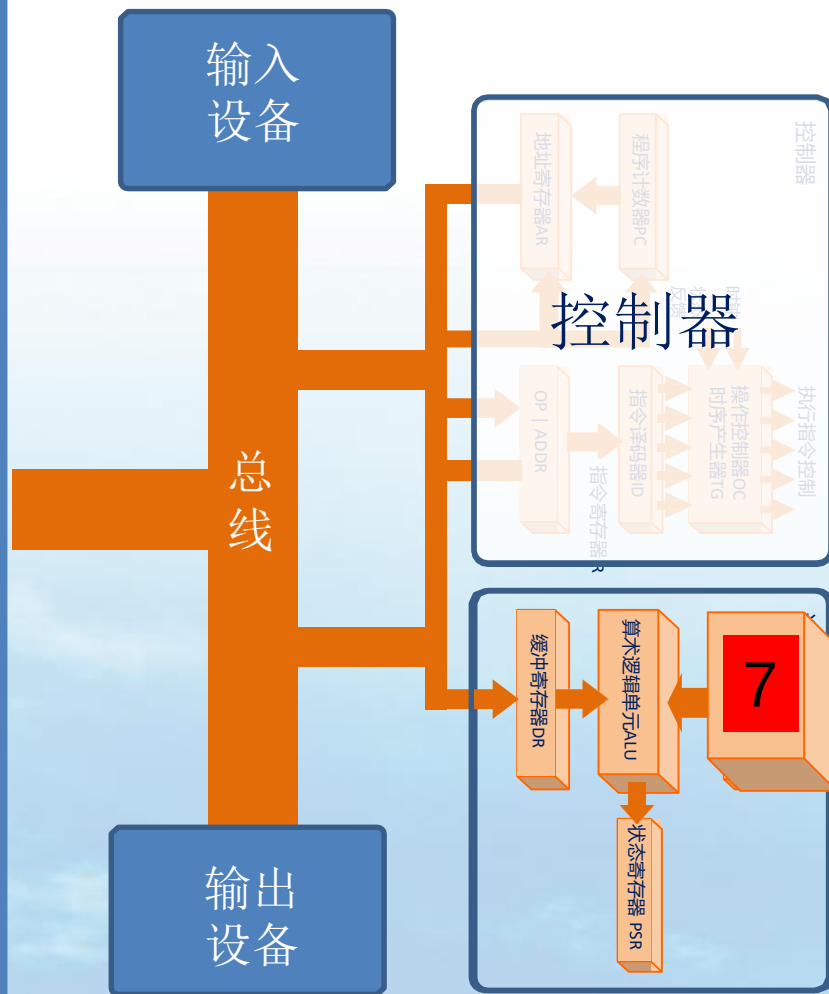
## 数据区

4

3

7

# 存储器





# 程序示例小结

- 程序执行

- 程序必须要经过编译才转换成CPU所能接受的指令;
- 一句程序有可能转换为多句指令;
- 在控制器的协调下连续、依次执行相应的指令;
- 程序执行过程是在内存中完成的;
- 程序在执行过程中, 在内存中的不同区域, 存放代码和相关的数据。



# 总结

- 冯·诺依曼计算机的思想及组成部分
  - 存储程序思想
- 命令及其执行
- 程序及其运行



## 进一步阅读文献

- 《计算机科学概论》（机械工业出版社）  
第5.1-5.2节（中/英）
- 《计算机科学概论》（人民邮电出版社）  
第2.1节（中/英）
- 《计算思维导论》第2.3节





# 作业一

- 1. 请把下列数转换成十进制数
  - 1)  $(101)_2$       2)  $(776)_8$       3)  $(BFE)_{16}$       4)  $(7A)_{16}$
  - 5)  $(0.1011)_2$       6)  $(1010.0101)_2$       7)  $(B7.A3)_{16}$
- 2. 请把下列十进制数分别转换成二进制和十六进制表示（结果位数适当即可，最高位无需补0）
  - 1) 43      2) 98      3) 15
- 3. 把下列实数转换成二进制表示（最高位无需补0，小数点后保留五位）
  - 1) 0.50      2) 0.26      3) 27.41      4) 101.101
- 4. 使用八位二进制补码计算下列表达式，其中 A 等于  $(18)_{10}$ ，B 等于  $(99)_{10}$ ，要求写出计算过程。
  - 1)  $A+B$       2)  $A-B$       3)  $(-A)-B$

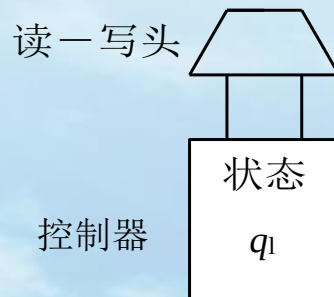


# 作业一

5.  $b$ 表示空格，机器状态集为 $\{q_1, q_2, q_3, q_4\}$ ， $q_4$ 表示停机状态，设带子上的输入信息是10100011，读入头位对准最右边第一个为1的方格，起始状态 $q_1$ 。指令集如下：

- |                             |                             |                             |
|-----------------------------|-----------------------------|-----------------------------|
| (1) $q_1 \ 0 \ 1 \ L \ q_2$ | (2) $q_1 \ 1 \ 0 \ L \ q_3$ | (3) $q_1 \ b \ b \ N \ q_4$ |
| (4) $q_2 \ 0 \ 0 \ L \ q_2$ | (5) $q_2 \ 1 \ 1 \ L \ q_2$ | (6) $q_2 \ b \ b \ N \ q_3$ |
| (7) $q_3 \ 0 \ 1 \ L \ q_2$ | (8) $q_3 \ 1 \ 0 \ L \ q_3$ | (9) $q_3 \ b \ b \ N \ q_4$ |

|     |   |   |   |   |   |   |   |   |   |   |   |   |   |     |
|-----|---|---|---|---|---|---|---|---|---|---|---|---|---|-----|
| ... | b | b | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | b | b | b | ... |
|-----|---|---|---|---|---|---|---|---|---|---|---|---|---|-----|



要求：写出规则执行的每一步的指令序列号及画出最终运行结果。



# 交作业方式

- 1. 每人都要交作业，杜绝抄袭，诚信做人 😊 ~
- 2. 把作业写在**纸上**！超过一页纸的自己装订好！ 🤔
- 3. 纸上写好**姓名**、**学号**！漏写的后果自负
- 4. 怎么交作业？津南校区的同学请课前或者课间交到讲台上即可，二学位同学请班长收好，统一交给助教，谢谢合作~
- 5. 本次作业10月28日上课交~



Thank You !



